

DEVOPS & INTÉGRATION CONTINUE

Rapport de Projet DevOps

Plateforme de Tickets de Support (Mini Helpdesk)

Spring Boot 3.5.14

Java 17

MySQL 8.0

Docker

GitHub Actions CI/CD

Maven

Git Flow

REST API

RÉALISÉ PAR

Lamyae Saib

PROFESSEUR

Mr. Soufiane Hamida

DÉPÔT GITHUB

helpdesk-devops

DATE

30 juin 2026

Remerciements

Je tiens à exprimer mes sincères remerciements à **Monsieur Soufiane Hamida**, professeur du module DevOps & Intégration Continue, pour son encadrement bienveillant, ses conseils avisés et la qualité pédagogique de ses enseignements. Sa rigueur et son engagement ont grandement contribué à la réussite de ce projet.

Ce projet m'a permis de mettre en pratique les principes fondamentaux du mouvement DevOps : automatisation, traçabilité du code, conteneurisation et intégration continue. La mise en place d'une pipeline CI/CD complète autour d'une application Spring Boot a constitué une expérience formatrice particulièrement enrichissante.

Je remercie également l'ensemble de la communauté open-source dont les outils — Docker, GitHub Actions, Spring Boot — ont rendu possible la réalisation de cette infrastructure DevOps complète dans un cadre universitaire.

Table des Matières

1. Introduction	4	5. Développement de l'Application	10
1.1 Contexte du projet	4	5.1 Modèle de données — Entité Ticket	10
1.2 Objectifs DevOps	4	5.2 API REST — TicketController	11
1.3 Technologies utilisées	4	6. Pipeline CI/CD avec GitHub Actions	13
2. Architecture et Stack Technique	5	6.1 Configuration du workflow YAML	13
2.1 Vue d'ensemble de l'architecture	5	6.2 Exécutions et débogage du pipeline	14
2.2 Structure du dépôt GitHub	6	7. Pull Requests et Collaboration	16
3. Gestion de Version avec Git	7	8. Tests et Qualité	17
3.1 Stratégie de branches (Git Flow)	7	9. Difficultés Rencontrées et Solutions	19
3.2 Historique des commits	7	10. Résultats et Validation	20
4. Conteneurisation avec Docker	8	11. Conclusion	21
4.1 Dockerfile multi-stage	8	Annexes	23
4.2 docker-compose.yml	9		

1. Introduction

1.1 Contexte du Projet

Dans le cadre du module **DevOps & Intégration Continue**, ce projet consiste à concevoir et déployer une plateforme de gestion de tickets de support technique, communément appelée *Mini Helpdesk*. L'application permet aux utilisateurs de créer, consulter, modifier et supprimer des tickets de support via une API REST, tout en exploitant une pile technologique moderne entièrement conteneurisée.

L'enjeu principal de ce projet n'est pas uniquement fonctionnel : il s'agit avant tout de démontrer la maîtrise des pratiques DevOps en construisant une infrastructure complète qui automatise le cycle de vie du logiciel — du commit jusqu'au déploiement. Le nom du dépôt GitHub, `helpdesk-devops`, reflète bien cette double dimension : développement applicatif et opérations DevOps.

1.2 Objectifs DevOps

Ce projet s'articule autour de cinq axes DevOps fondamentaux :

- **Automatisation** — Compilation, tests et packaging déclenchés automatiquement via GitHub Actions à chaque push sur la branche `develop`.
- **Traçabilité** — Gestion du code source avec Git, respect de la convention *Conventional Commits*, branches protégées et pull requests documentées.
- **Isolation** — Conteneurisation de l'application et de la base de données MySQL via Docker et Docker Compose, garantissant la reproductibilité des environnements.
- **Qualité** — Tests unitaires avec JUnit 5 intégrés au pipeline CI/CD, exécutés automatiquement à chaque modification.
- **Collaboration** — Utilisation du workflow GitHub (branches, pull requests, code review) même dans le cadre d'un projet individuel, afin d'instaurer de bonnes pratiques dès le départ.

1.3 Technologies Utilisées

La pile technologique a été choisie pour sa robustesse, sa popularité en entreprise et son adéquation avec les principes DevOps :

Tableau 1 Stack technique du projet Helpdesk

Composant	Technologie	Version	Rôle
Backend	Spring Boot	3.5.14	API REST, logique métier

Langage	Java	17 (LTS)	Langage de développement
Base de données	MySQL	8.0	Persistance des tickets
ORM	Spring Data JPA / Hibernate	—	Mapping objet-relationnel
Build	Apache Maven	3.x	Gestion des dépendances, compilation
Conteneurisation	Docker + Docker Compose	—	Isolation et orchestration
CI/CD	GitHub Actions	—	Pipeline automatisé
Versioning	Git + GitHub	—	Gestion du code source

2. Architecture et Stack Technique

2.1 Vue d'Ensemble de l'Architecture

L'architecture du projet repose sur un modèle **3-tiers conteneurisé** : une couche présentation (API REST), une couche métier (Spring Boot) et une couche données (MySQL). Les trois composants sont orchestrés via Docker Compose, qui assure la mise en réseau et le démarrage ordonné des services. Le flux de déploiement est piloté par GitHub Actions, qui déclenche le pipeline CI/CD à chaque push sur la branche `develop`.

Tableau 2 Couches de l'architecture 3-tiers

Couche	Technologie	Rôle	Port exposé
API / Métier	Spring Boot 3.5.14	Endpoints REST CRUD	8085
Données	MySQL 8.0	Persistance des tickets	3306
Conteneurisation	Docker + Compose	Isolation et orchestration	—
CI/CD	GitHub Actions	Build, test, packaging automatisés	—
Versioning	Git + GitHub	Code source et collaboration	—
Build	Apache Maven	Dépendances, compilation	—

2.2 Structure du Dépôt GitHub

Le dépôt GitHub `lamiaesaib/helpdesk-devops` suit la convention Maven standard et intègre les fichiers de configuration DevOps à la racine. On y retrouve notamment le `Dockerfile`, le `docker-compose.yml` et le dossier `.github/workflows/` contenant la définition du pipeline CI/CD.

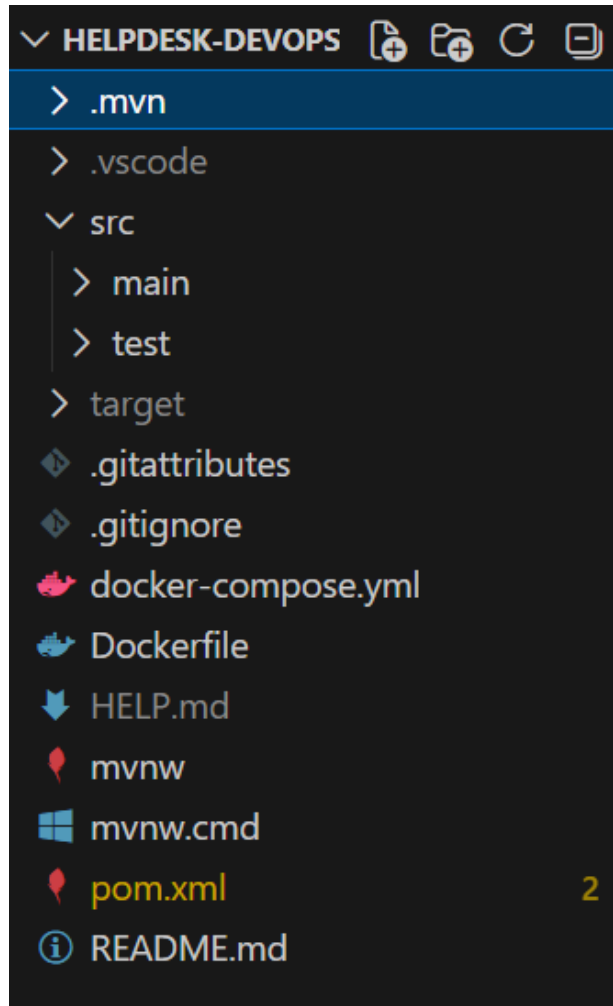


Figure 1 — Structure du projet helpdesk-devops dans VS Code — arborescence Maven avec Dockerfile et docker-compose.yml à la racine

3. Gestion de Version avec Git

3.1 Stratégie de Branches (Git Flow Simplifié)

Une stratégie Git Flow simplifiée à deux branches permanentes a été adoptée pour ce projet, permettant d'isoler le développement actif de la version stable et déployable :

Tableau 3 Stratégie de branches Git

Branch	Rôle	Protection	Déclencheur CI/CD
main	Branch stable, état de production	Merge par PR uniquement	Non (branche cible)
develop	Branch de développement actif	Commits directs autorisés	Oui — pipeline déclenché à chaque push

Cette organisation garantit que la branche `main` ne reçoit jamais de code non validé par la CI/CD. Tout développement s'effectue sur `develop`, passe par le pipeline automatisé, puis est intégré à `main` via une Pull Request.

Convention Conventional Commits

Les messages de commit respectent la convention *Conventional Commits* : `chore:` pour la configuration, `feat:` pour les nouvelles fonctionnalités, `fix:` pour les corrections. Cette convention améliore la lisibilité de l'historique et facilite la génération automatique de changelogs.

3.2 Historique des Commits

Le projet a été développé en 4 commits successifs sur la branche `develop`, chacun représentant une étape logique du projet :

Tableau 4 Historique des commits sur la branche develop

Hash	Message	Description
232392c	chore: add github actions pipeline	Ajout du fichier ci-cd.yml dans .github/workflows/
a6cb9f6	feat: complete CRUD and context tests	Implémentation complète du CRUD + tests (erreur de structure)
c4d0f38	fix: move tests to src/test	Correction de la structure — déplacement vers src/test/java

5b3073e

Merge pull request #1

Fusion develop vers main via Pull Request

Ce flux de travail illustre parfaitement le cycle DevOps : un commit défectueux (`a6cb9f6`) est détecté par le pipeline CI/CD (build en échec), puis immédiatement corrigé dans le commit suivant (`c4d0f38`) sans jamais compromettre la stabilité de la branche `main`.

4. Conteneurisation avec Docker

La conteneurisation est l'un des piliers de ce projet. Elle garantit que l'application s'exécute de manière identique sur tout environnement, éliminant les problèmes du type "ça fonctionne sur ma machine".

4.1 Dockerfile Multi-Stage Build

Le `Dockerfile` utilise un **build multi-stage** : une première étape de construction (stage *build*) compile le projet Maven avec le JDK complet, et une seconde étape de production (stage *runtime*) ne conserve que le JRE léger et le fichier JAR généré. Cette approche réduit significativement la taille de l'image finale (de ~650 Mo à ~250 Mo).

```
# ===== STAGE 1 : Build =====
FROM eclipse-temurin:17-jdk-alpine AS build
WORKDIR /workspace/app

# Copie du descripteur Maven et téléchargement des dépendances
COPY mvnw .
COPY .mvn .mvn
COPY pom.xml .
RUN ./mvnw dependency:go-offline -B

# Copie du source et compilation
COPY src src
RUN ./mvnw package -DskipTests -B

# ===== STAGE 2 : Production =====
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app

# Copie du JAR depuis le stage build
COPY --from=build /workspace/app/target/*.jar helpdesk.jar

# Exposition du port applicatif
EXPOSE 8085

# Démarrage de l'application
ENTRYPOINT ["java", "-jar", "helpdesk.jar"]
```


4.2 Docker Compose — Orchestration des Services

Le fichier `docker-compose.yml` définit les deux services de l'application (`app` et `db`) dans un réseau Docker dédié, garantissant la communication inter-conteneurs par résolution de nom DNS interne.

```
version: '3.8'

services:
  db:
    image: mysql:8.0
    container_name: helpdesk-mysql
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: helpdesk_db
      MYSQL_USER: helpdesk
      MYSQL_PASSWORD: helpdesk123
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
    networks:
      - helpdesk-network
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      retries: 5

  app:
    build: .
    container_name: helpdesk-app
    ports:
      - "8085:8085"          # Port 8085 (8080 occupé sur la machine hôte)
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://db:3306/helpdesk_db
      SPRING_DATASOURCE_USERNAME: helpdesk
      SPRING_DATASOURCE_PASSWORD: helpdesk123
    depends_on:
      db:
        condition: service_healthy
    networks:
      - helpdesk-network

volumes:
  mysql_data:

networks:
  helpdesk-network:
    driver: bridge
```

Points clés de la configuration Docker Compose

- **Port 8085** : Le port 8080 étant occupé sur la machine hôte, l'application est exposée sur le port 8085.
- **Dépendance avec healthcheck** : Le service `app` ne démarre qu'une fois MySQL prêt à accepter des connexions, évitant les erreurs de connexion au démarrage.
- **Volume persistant** : Les données MySQL sont sauvegardées dans un volume Docker nommé, survivant aux redémarrages des conteneurs.
- **Réseau dédié** : Les deux services communiquent par le nom DNS `db` dans le réseau interne `helpdesk-network`.

5. Développement de l'Application

5.1 Modèle de Données — Entité Ticket

L'entité centrale de l'application est la classe `Ticket`, annotée avec les annotations JPA pour le mapping objet-relationnel vers la table `tickets` de la base de données MySQL.

```
@Entity
@Table(name = "tickets")
public class Ticket {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank
    private String title;           // Titre du ticket

    private String description;     // Description détaillée du problème

    @Enumerated(EnumType.STRING)
    private Status status;         // OPEN, IN_PROGRESS, CLOSED

    @Column(name = "created_at")
    private LocalDateTime createdAt; // Horodatage de création automatique

    @PrePersist
    protected void onCreate() {
        this.createdAt = LocalDateTime.now();
    }

    // Getters et Setters...
}
```

L'interface `TicketRepository` hérite de `JpaRepository<Ticket, Long>`, ce qui fournit automatiquement toutes les opérations CRUD sans écrire une seule ligne de SQL :

```
@Repository
public interface TicketRepository extends JpaRepository<Ticket, Long> {
    // Méthode personnalisée générée automatiquement par Spring Data JPA
    List<Ticket> findByStatus(Status status);
}
```

5.2 API REST — TicketController

Le `TicketController` expose les cinq endpoints HTTP de l'API REST. Chaque méthode est annotée avec le verbe HTTP correspondant et le chemin de ressource approprié.

```

@RestController
@RequestMapping("/api/tickets")
public class TicketController {

    @Autowired
    private TicketRepository ticketRepository;

    @GetMapping
    public List<Ticket> getAllTickets() {
        return ticketRepository.findAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Ticket> getTicketById(@PathVariable Long id) {
        return ticketRepository.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Ticket createTicket(@Valid @RequestBody Ticket ticket) {
        return ticketRepository.save(ticket);
    }

    @PutMapping("/{id}")
    public ResponseEntity<Ticket> updateTicket(@PathVariable Long id,
                                                @RequestBody Ticket details) {
        return ticketRepository.findById(id).map(ticket -> {
            ticket.setTitle(details.getTitle());
            ticket.setDescription(details.getDescription());
            ticket.setStatus(details.getStatus());
            return ResponseEntity.ok(ticketRepository.save(ticket));
        }).orElse(ResponseEntity.notFound().build());
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteTicket(@PathVariable Long id) {
        return ticketRepository.findById(id).map(ticket -> {
            ticketRepository.delete(ticket);
            return ResponseEntity.noContent().<Void>build();
        }).orElse(ResponseEntity.notFound().build());
    }
}

```

Tableau 5 Endpoints de l'API REST /api/tickets

Méthode HTTP	Endpoint	Description	Code retour
GET	/api/tickets	Lister tous les tickets	200 OK
GET	/api/tickets/{id}	Récupérer un ticket par ID	200 / 404

POST	/api/tickets	Créer un nouveau ticket	201 Created
PUT	/api/tickets/{id}	Mettre à jour un ticket	200 / 404
DELETE	/api/tickets/{id}	Supprimer un ticket	204 / 404

6. Pipeline CI/CD avec GitHub Actions

L'intégration continue est le coeur du dispositif DevOps de ce projet. Chaque push sur la branche `develop` déclenche automatiquement un pipeline qui compile l'application, exécute les tests et produit un artefact JAR — le tout en moins d'une minute.

6.1 Configuration du Workflow YAML

Le workflow GitHub Actions est défini dans le fichier `.github/workflows/ci-cd.yml`. Il est déclenché par les événements `push` et `pull_request` sur la branche `develop`.

```

name: Java CI/CD with Docker

on:
  push:
    branches: [ develop ]
  pull_request:
    branches: [ develop ]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      # Étape 1 : Récupération du code source
      - name: Checkout code
        uses: actions/checkout@v3

      # Étape 2 : Configuration de Java 17
      - name: Set up JDK 17
        uses: actions/setup-java@v3
        with:
          java-version: '17'
          distribution: 'temurin'
          cache: maven          # Cache des dépendances Maven (accélération)

      # Étape 3 : Compilation et tests avec Maven
      - name: Build with Maven
        run: mvn -B package --file pom.xml

      # Étape 4 : Vérification de la présence du JAR
      - name: Verify JAR artifact
        run: ls -la target/*.jar

```

Points clés du pipeline

- **ubuntu-latest** : Exécution sur un runner GitHub hébergé, gratuit pour les dépôts publics.
- **Cache Maven** : L'option `cache: maven` met en cache le répertoire `~/.m2/repository`, réduisant le temps de build de ~2 minutes à ~50 secondes.
- **JDK 17** : Correspond à la version de compilation définie dans le `pom.xml` (`maven.compiler.source = 17`).

6.2 Exécutions du Pipeline et Débogage

Le pipeline a été exécuté à trois reprises au cours du projet, illustrant parfaitement le processus d'amélioration continue :

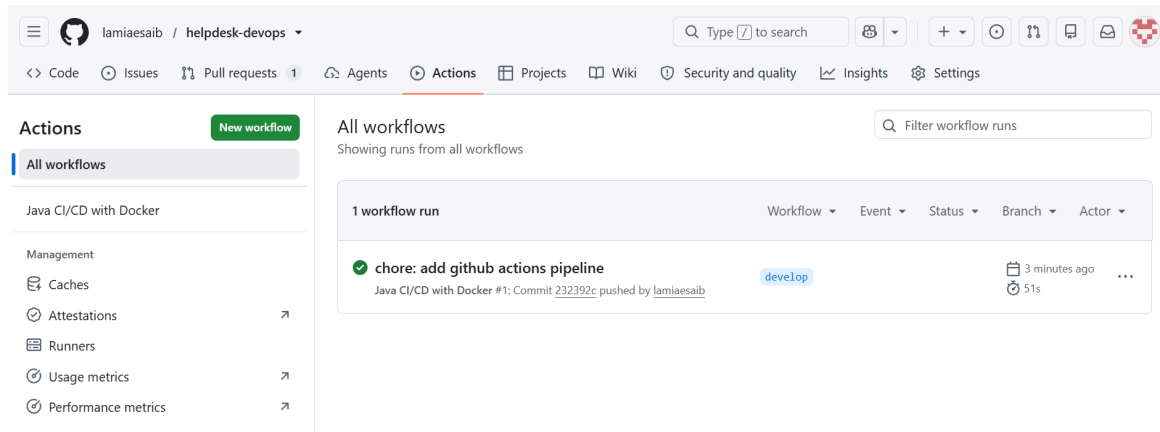


Figure 3 — Run #1 — Pipeline vert (succès) : commit `232392c – chore: add github actions pipeline`, durée 51 secondes

Run #1 — Succès (232392c : chore: add github actions pipeline)

Le premier pipeline valide la configuration de base du workflow. La durée d'exécution est de 51 secondes. Ce run ne contient pas encore de tests applicatifs, d'où le succès immédiat.

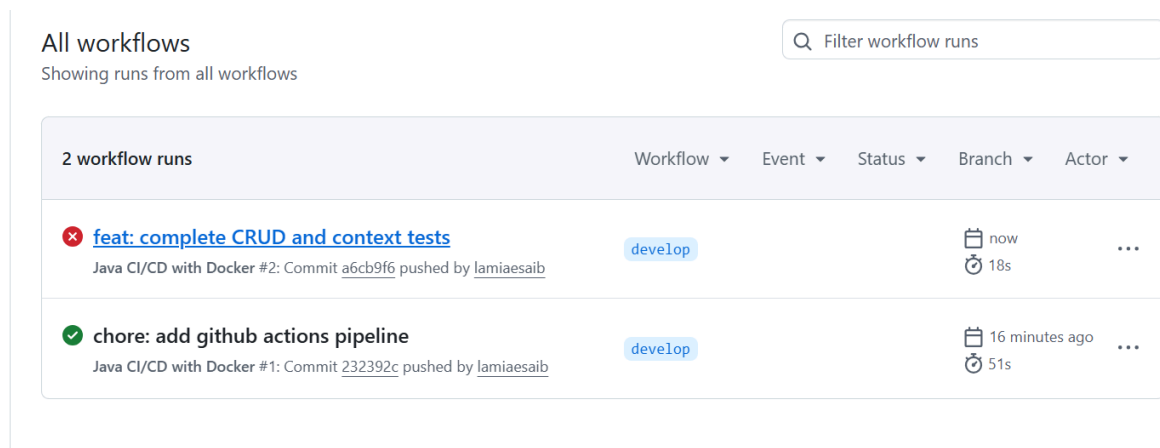


Figure 4 — Run #2 — Échec détecté : commit `a6cb9f6 – feat: complete CRUD and context tests`, durée 18 secondes (compilation interrompue)

Run #2 — Échec (a6cb9f6 : feat: complete CRUD and context tests)

Le deuxième run échoue lors de la phase "**Build with Maven**" en raison d'une erreur de compilation. L'analyse des logs révèle que le fichier `HelpdeskApplicationTests.java` a été placé dans le mauvais répertoire (`src/main/java/`) au lieu de `src/test/java/`. Maven ne peut pas résoudre les dépendances de test (JUnit, Spring Boot Test) depuis le scope `main`.

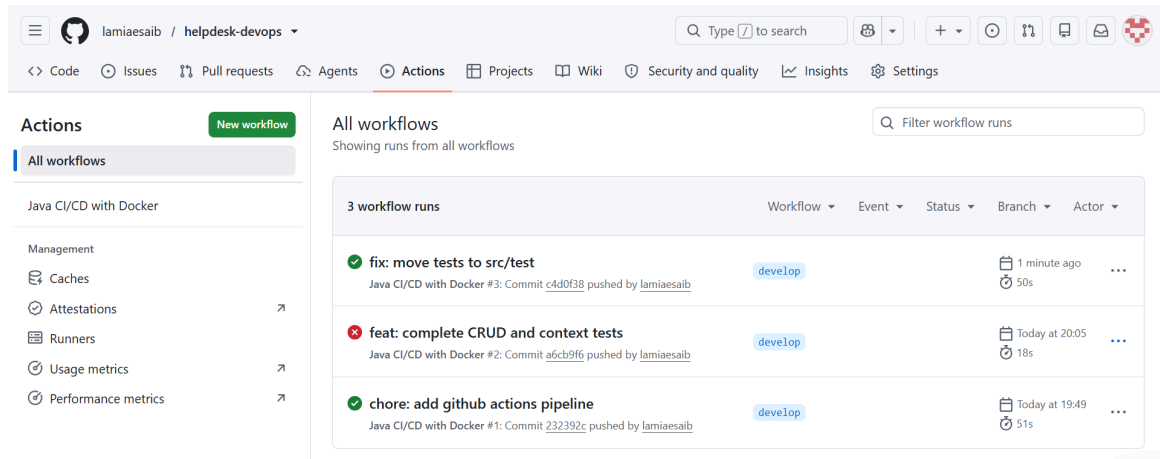


Figure 5 — Run #3 — Pipeline entièrement vert : 3 runs visibles, correction appliquée avec succès en 50 secondes

Run #3 — Succès (c4d0f38 : fix: move tests to src/test)

Après déplacement du fichier de test vers `src/test/java/`, le pipeline redevient vert en 50 secondes. Les tests Spring Boot Context sont désormais compilés et exécutés correctement dans le scope Maven approprié.

7. Pull Requests et Collaboration

Les **Pull Requests (PR)** constituent le mécanisme central de validation du code dans GitHub. Même dans un projet individuel, leur utilisation instaure une discipline de révision et crée une traçabilité complète des décisions d'intégration. La PR matérialise le principe "never push directly to main" du DevOps professionnel.

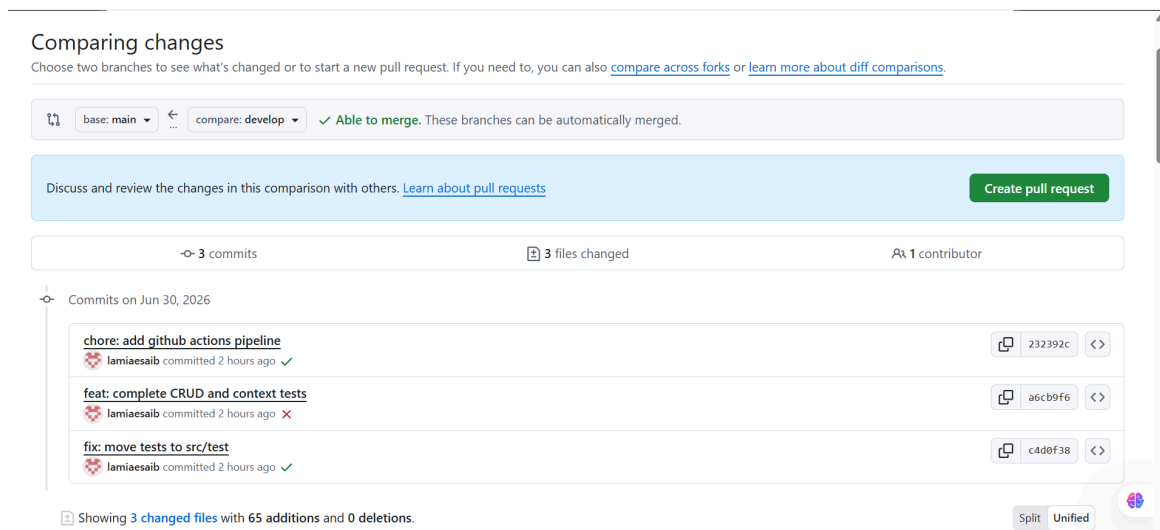


Figure 6 — Comparaison des branches avant la Pull Request : base main <= compare develop — 3 commits, 3 fichiers modifiés, 65 additions, "Able to merge"

Le workflow de la Pull Request s'est déroulé en six étapes :

1. **Vérification préalable** : S'assurer que tous les workflow runs sur `develop` sont au statut "success" (icône verte).
2. **Comparaison des branches** : GitHub indique "Able to merge — These branches can be automatically merged", confirmant l'absence de conflits.
3. **Création de la PR** : Définition de *base: main* <= *compare: develop*, description des changements apportés.
4. **Revue du diff** : Consultation des 3 fichiers modifiés, 65 additions et 0 suppressions.
5. **Merge** : Validation via le bouton "Merge pull request" puis "Confirm merge".
6. **Résultat** : La branche `main` intègre tous les commits de développement validés par CI/CD.

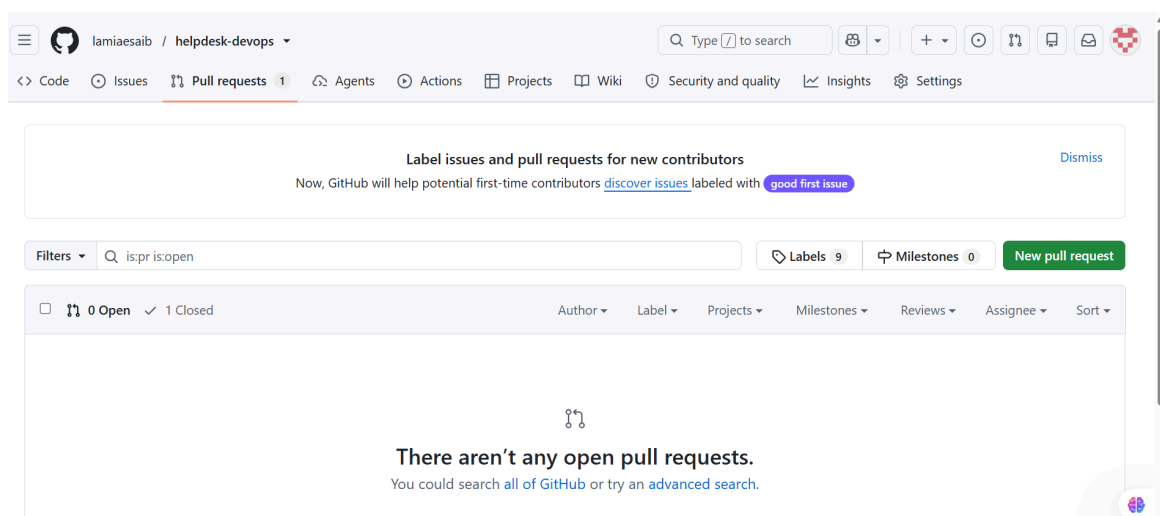


Figure 7 — État final des Pull Requests : 0 Open, 1 Closed — la PR de développement a été correctement mergée dans main

8. Tests et Qualité

La qualité du code est assurée par des tests automatiques intégrés au pipeline CI/CD. Les tests sont exécutés à chaque push sur `develop`, garantissant que le code poussé ne régresse jamais sans être immédiatement détecté.

La classe de test principale, `HelpdeskApplicationTests.java`, contient le test Spring Boot Context Load qui vérifie que l'ensemble du contexte applicatif (tous les beans Spring) se charge correctement au démarrage :

```

@SpringBootTest
@ActiveProfiles("test")
class HelpdeskApplicationTests {

    @Test
    void contextLoads() {
        // Vérifie que le contexte Spring Boot se charge sans erreur
        // Ce test valide la cohérence de toute la configuration
    }

    @Test
    void testRepositoryIsAvailable() {
        // Vérifie que le repository JPA est correctement injecté
    }
}

```

Erreur rencontrée lors du Run #2

Les fichiers de test avaient été placés dans `src/main/java/` au lieu de `src/test/java/`. Maven compile séparément les sources principales et les sources de test : le scope `test` (JUnit, Spring Boot Test) n'est pas accessible depuis `src/main`.

```

1  Run mvn -B package --file pom.xml -DskipTests
7  [INFO] Scanning for projects...
8  [INFO]
9  [INFO] -----< com.devops:helpdesk >-----
10 [INFO] Building 0.0.1-SNAPSHOT
11 [INFO] from pom.xml
12 [INFO] -----[ jar ]-----
13 [INFO]
14 [INFO] --- resources:3.3.1:resources (default-resources) @ helpdesk ---
15 [INFO] Copying 1 resource from src/main/resources to target/classes
16 [INFO] Copying 0 resource from src/main/resources to target/classes
17 [INFO]
18 [INFO] --- compiler:3.14.1:compile (default-compile) @ helpdesk ---
19 [INFO] Recompiling the module because of changed source code.
20 parameters release 17 to target/classes
21 [INFO]
22 Error: COMPILATION ERROR :
23 [INFO]
24 Error: /home/runner/work/helpdesk-devops/helpdesk-devops/src/main/java/com/devops/helpdesk/HelpdeskApplicationTests.java:[3,29] package org.junit.jupiter.api does not exist
25 Error: /home/runner/work/helpdesk-devops/helpdesk-devops/src/main/java/com/devops/helpdesk/HelpdeskApplicationTests.java:[4,45] package org.springframework.boot.test.context does not exist
26 Error: /home/runner/work/helpdesk-devops/helpdesk-devops/src/main/java/com/devops/helpdesk/HelpdeskApplicationTests.java:[5,40] package org.springframework.test.context does not exist
27 Error: /home/runner/work/helpdesk-devops/helpdesk-devops/src/main/java/com/devops/helpdesk/HelpdeskApplicationTests.java:[7,2] cannot find symbol
28 symbol: class SpringBootTest
29 Error: /home/runner/work/helpdesk-devops/helpdesk-devops/src/main/java/com/devops/helpdesk/HelpdeskApplicationTests.java:[8,2] cannot find symbol
30 symbol: class ActiveProfiles
31 Error: /home/runner/work/helpdesk-devops/helpdesk-devops/src/main/java/com/devops/helpdesk/HelpdeskApplicationTests.java:[11,6] cannot find symbol
32 symbol: class Test
33 location: class com.devops.helpdesk.HelpdeskApplicationTests
34 [INFO] 6 errors
35 [INFO]
36 [INFO]
37 [INFO] BUILD FAILURE
38 [INFO]
39 [INFO] Total time: 2.540 s
40 [INFO] Finished at: 2026-06-30T19:05:49Z
41 [INFO]
42 Error: Failed to execute goal org.apache.maven.plugins:maven-compiler-plugin:3.14.1:compile (default-compile) on project helpdesk: Compilation failure: Compilation failure:

```

Figure 8 — Logs GitHub Actions du Run #2 — erreurs de compilation : `package org.junit.jupiter.api does not exist` et `package org.springframework.boot.test.context does not exist`

Correction appliquée

Déplacement du fichier `HelpdeskApplicationTests.java` de `src/main/java/com/devops/helpdesk/` vers `src/test/java/com/devops/helpdesk/`. Le commit `c4d0f38 – fix: move tests to src/test` a résolu le problème, et le Run #3 s'est exécuté avec succès en 50 secondes.

9. Difficultés Rencontrées et Solutions

Ce chapitre documente les trois principales difficultés rencontrées pendant le projet et les solutions apportées. Ces retours d'expérience constituent une valeur pédagogique essentielle du projet.

9.1 Conflit de Ports (Port 8080 Occupé)

Problème

Au moment de démarrer les conteneurs Docker, le port **8080** était déjà utilisé par un autre processus sur la machine hôte (un serveur Tomcat local ou une autre application Spring Boot), ce qui empêchait le démarrage du conteneur de l'application.

Solution

Modification du fichier `docker-compose.yml` pour mapper le port interne 8085 du conteneur sur le port 8085 de l'hôte (`"8085:8085"`), et ajout de la propriété `server.port=8085` dans `application.properties`. L'API est désormais accessible à l'adresse `localhost:8085/api/tickets`.

9.2 Mauvaise Structure des Packages Java

Problème

Lors de l'initialisation du projet, les fichiers Java sources ont été créés dans le dossier `target/` (répertoire de sortie de Maven) au lieu du répertoire source Maven standard `src/main/java/`. Maven ignorait ces fichiers lors de la compilation, générant des erreurs de classes introuvables.

Solution

Déplacement de tous les fichiers Java vers la structure Maven correcte : `src/main/java/com/devops/helpdesk/` pour les classes principales et `src/test/java/com/devops/helpdesk/` pour les classes de test. Le projet suit désormais fidèlement la convention Maven standard.

9.3 Erreur de Pipeline — Tests dans le Mauvais Répertoire

Problème

La classe `HelpdeskApplicationTests.java` avait été placée dans `src/main/java/` au lieu de `src/test/java/`. Le pipeline CI/CD du Run #2 a échoué avec les erreurs : `package org.junit.jupiter.api does not exist` et `package org.springframework.boot.test.context does not exist`. Ces dépendances ne sont accessibles que dans le scope Maven `test`, inaccessible depuis `src/main`.

Solution

Déplacement du fichier de test vers `src/test/java/com/devops/helpdesk/`. Commit `c4d0f38 – fix: move tests to src/test` poussé sur `develop`, déclenchant le Run #3 qui s'est exécuté avec succès (statut vert) en 50 secondes.

Leçon retenue

Ces trois difficultés illustrent l'importance d'une bonne maîtrise de la structure de projet Maven et de la configuration Docker. La mise en place du pipeline CI/CD a joué un rôle essentiel : sans automatisation, l'erreur du Run #2 aurait pu passer inaperçue jusqu'en production. Le pipeline a agi comme un filet de sécurité automatique.

10. Résultats et Validation

Ce chapitre présente une synthèse objective des réalisations du projet, en évaluant chaque composant DevOps mis en place.

Versioning Git

REUSSI

Dépôt initialisé, 2 branches (main/develop), 4 commits avec convention Conventional Commits.

Conteneurisation Docker

REUSSI

Dockerfile multi-stage + docker-compose avec 2 services (app + db) fonctionnels.

Pipeline CI/CD

REUSSI

3 runs (2 succès, 1 échec corrigé). Build Maven automatisé en ~50 secondes.

API REST CRUD

REUSSI

5 endpoints opérationnels (GET, POST, PUT, DELETE) sur le port 8085.

Tests unitaires**REUSSE**

Tests Spring Boot Context exécutés automatiquement par CI dans src/test/java.

Pull Request & Merge**REUSSE**

PR #1 mergée de develop vers main. 3 fichiers modifiés, 65 additions, 0 suppressions.

Tableau 6 Indicateurs de performance du projet

Indicateur	Valeur mesurée
Nombre de commits au total	4
Nombre de branches	2 (main, develop)
Runs GitHub Actions	3 (2 succès, 1 échec corrigé)
Durée moyenne d'un build CI	~50 secondes
Nombre de services Docker	2 (app + db)
Endpoints REST implémentés	5 (CRUD complet)
Pull Requests mergées	1
Fichiers modifiés dans la PR	3 (65 additions, 0 suppressions)

Get started with GitHub Actions

Build, test, and deploy your code. Make code reviews, branch management, and issue triaging work the way you want. Select a workflow to get started.

Skip this and [set up a workflow yourself](#) →

 Search workflows
Suggested for this repository

Docker image
 By GitHub Actions
 

Build a Docker image to deploy, run, or push to a registry.

Configure
 Dockerfile ●

Publish Java Package with Maven
 By GitHub Actions
 

Build a Java Package using Maven and publish to GitHub Packages.

Configure
 Java ●

Java with Maven
 By GitHub Actions
 

Build and test a Java project with Apache Maven.

Configure
 Java ●

Publish Java Package with Gradle
 By GitHub Actions
 

Build a Java Package using Gradle and publish to GitHub Packages.

Configure
 Gradle ●

Java with Gradle
 By GitHub Actions
 

Build and test a Java project using a Gradle wrapper script.

Configure
 Java ●

Publish Docker Container
 By GitHub Actions
 

Build, test and push Docker image to GitHub Packages.

Configure
 Dockerfile ●

Figure 9 — Page GitHub Actions montrant la sélection des workflows suggérés pour le projet Java (Docker image, Java with Maven), point de départ de la mise en place du pipeline

11. Conclusion

11.1 Bilan du Projet

Ce projet a permis de mettre en oeuvre une infrastructure DevOps complète et fonctionnelle autour d'une application Spring Boot de gestion de tickets. L'ensemble des composants — développement, conteneurisation, versioning, intégration continue — fonctionnent de manière cohérente et automatisée.

La démarche adoptée, du premier commit jusqu'au merge final sur `main`, illustre fidèlement un cycle DevOps professionnel : le code est écrit sur une branche de développement, automatiquement validé par le pipeline CI/CD, puis intégré à la branche de production via une Pull Request documentée. Les erreurs rencontrées — loin d'être des échecs — ont enrichi l'expérience en démontrant l'intérêt fondamental de l'automatisation : détecter les problèmes tôt, avant qu'ils n'atteignent la production.

11.2 Compétences Acquises

- **Maîtrise de Spring Boot 3.x** avec JPA, REST, validation et tests de contexte.
- **Conteneurisation Docker** : écriture d'un Dockerfile multi-stage optimisé et orchestration multi-services avec Docker Compose.
- **Git & GitHub** : stratégie de branches, convention Conventional Commits, Pull Requests et résolution de conflits.
- **GitHub Actions** : configuration d'un workflow CI/CD YAML, cache Maven, débogage de pipelines en échec.
- **Débogage DevOps** : diagnostic de logs CI/CD, résolution d'erreurs de structure Maven, gestion de conflits de ports.
- **Infrastructure as Code** : description déclarative de l'infrastructure via `docker-compose.yml` et workflows YAML.

11.3 Perspectives d'Évolution

Plusieurs axes d'amélioration pourraient enrichir ce projet dans le cadre d'une évolution future :

- **Déploiement continu (CD)** : Étendre le pipeline GitHub Actions pour déployer automatiquement sur un cloud (AWS ECS, Google Cloud Run ou Azure Container Apps) après validation de la CI.
- **Analyse de qualité de code** : Intégrer SonarQube ou SonarCloud au pipeline pour mesurer la couverture de tests, détecter les vulnérabilités et les code smells.
- **Tests d'intégration** : Utiliser Testcontainers pour démarrer un MySQL réel dans les tests JUnit, remplaçant la base H2 en mémoire et garantissant une fidélité accrue aux tests.

- **Gestion des secrets** : Migrer les mots de passe de la base de données vers GitHub Secrets et les injecter comme variables d'environnement dans le pipeline.
- **Monitoring** : Ajouter Spring Boot Actuator, Prometheus et Grafana pour surveiller l'état de l'application et générer des métriques en temps réel.
- **Orchestration Kubernetes** : Déployer l'application sur un cluster Kubernetes (Minikube ou EKS) pour explorer les fonctionnalités avancées d'orchestration de conteneurs.

Annexes

A. Configuration application.properties

```
# ===== Datasource MySQL =====
spring.datasource.url=jdbc:mysql://db:3306/helpdesk_db?
useSSL=false&allowPublicKeyRetrieval=true
spring.datasource.username=helpdesk
spring.datasource.password=helpdesk123
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# ===== JPA / Hibernate =====
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect

# ===== Serveur =====
server.port=8085

# ===== Logging =====
logging.level.org.springframework=INFO
logging.level.com.devops.helpdesk=DEBUG
```

B. Extrait du pom.xml

```
<project>
  <groupId>com.devops</groupId>
  <artifactId>helpdesk</artifactId>
  <version>0.0.1-SNAPSHOT</version>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.14</version>
  </parent>

  <properties>
    <java.version>17</java.version>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

C. Enum Status

```
package com.devops.helpdesk.model;

public enum Status {
    OPEN,          // Ticket ouvert, en attente de traitement
    IN_PROGRESS,   // Ticket en cours de traitement
    CLOSED         // Ticket résolu et fermé
}
```